

Superfecta Prediction - Quick Start Guide

A machine learning system for predicting horse racing superfecta payouts using market data and advanced statistical modeling.

Table of Contents

- Overview
- Prerequisites
- Installation
- Data Requirements
- Quick Start (5 Steps)
- Detailed Workflow
- Live Predictions
- Troubleshooting
- Advanced Usage

Overview

This system uses machine learning to improve upon traditional Harville probability models for superfecta betting. It learns market biases and inefficiencies to provide more accurate payout predictions.

Key Features:

- Historical data analysis and model training
- Real-time NYRA API integration
- Anti-overfitting hyperparameter optimization
- Comprehensive bias analysis and corrections
- Production-ready prediction pipeline

Prerequisites

- **Python 3.8+** (recommended: 3.9 or 3.10)
- **Database access** with historical horse racing data
- **4GB+ RAM** (8GB+ recommended for large datasets)
- **Internet connection** for live NYRA API predictions

Installation

1. Clone/Download the Project

Shell

```
git clone <your-repository-url>  
cd superfecta-prediction-system
```

2. Create Virtual Environment (Recommended)

Shell

```
# Create virtual environment  
python -m venv venv  
  
# Activate virtual environment  
# On Windows:  
venv\Scripts\activate  
# On macOS/Linux:  
source venv/bin/activate
```

3. Install Required Packages

Shell

```
pip install --upgrade pip
```

Core Dependencies:

Shell

```
# Data Processing  
pip install pandas>=1.5.0  
pip install numpy>=1.21.0  
  
# Machine Learning  
pip install xgboost>=1.6.0  
pip install scikit-learn>=1.1.0  
  
# Hyperparameter Optimization  
pip install optuna>=3.0.0
```

```
# Visualization
pip install matplotlib>=3.5.0
pip install seaborn>=0.11.0

# API Integration
pip install requests>=2.28.0

# Numerical Computing
pip install scipy>=1.9.0

# Progress Bars
pip install tqdm>=4.64.0
```

All at once:

```
Shell

pip install pandas numpy xgboost scikit-learn optuna matplotlib
seaborn requests scipy tqdm
```

4. Verify Installation

```
Python

python -c "import pandas, numpy, xgboost, sklearn, optuna;
print('✅ All packages installed successfully!')"
```

Data Requirements

Historical Racing Data Schema

Your database should contain these tables with superfecta data:

Required Tables:

- **start** - Horse entries and odds

- `race` - Race details and conditions
- `exotic_payoff` - Superfecta payout information

Key Fields Needed:

SQL

```
-- start table
registration_number, program_number, odds, scratch_indicator,
position_at_start, official_position, post_position

-- race table
track_id, race_date, race_number, distance_id, surface,
course_type, purse_usd, post_time, track_condition,
number_of_runners

-- exotic_payoff table
winning_numbers, number_of_tickets_bet, total_pool,
payoff_amount, wager_type
```

Quick Start (5 Steps)

Step 1: Extract Training Data

Shell

```
# Run the SQL query to extract historical data
# Use SQLQuery_major_tracks.sql in your database
# Export results to CSV format
```

Expected output: `historical_data.csv` with 50,000+ race records

Step 2: Train Initial Model

Shell

```
python train_test_residual.py --csv historical_data.csv --output
initial_results.csv
```

What this does:

- Processes raw data into race-level features
- Trains XGBoost model with default parameters
- Creates visualizations showing model performance
- Saves trained model as `log_residual_model_normalized.json`

Step 3: Optimize Hyperparameters (Optional but Recommended)

Shell

```
# For major tracks (more conservative):
python tune_residual.py --csv historical_data.csv
--major-tracks-only --trials 50

# For all tracks (standard):
python tune_residual.py --csv historical_data.csv --trials 100
```

What this does:

- Runs hyperparameter optimization with anti-overfitting strategies
- Saves optimized model (better performance than Step 2)
- Creates optimization history plots

Step 4: Test Single Race Prediction

Shell

```
# Create a test file with single race data (same format as
training data)
python predict_single_race.py --input test_race.csv --model
log_residual_model_normalized.json --output predictions.csv
```

Step 5: Live Predictions (NYRA API)

Shell

```
python model_main.py --model log_residual_model_normalized.json
--output live_predictions.csv
```

Interactive mode: Select race cards and specific races **Automated mode:** Add `--auto-select` flag

Detailed Workflow

Phase 1: Data Preparation

Shell

```
# 1. Extract historical data using SQL query
# Run SQLQuery_major_tracks.sql in your database tool
# Export to CSV: historical_data.csv

# 2. Verify data quality
python -c "
import pandas as pd
df = pd.read_csv('historical_data.csv')
print(f'Records: {len(df):,}')
print(f'Date range: {df.race_date.min()} to {df.race_date.max()}')
print(f'Tracks: {df.track_id.unique()}')
print(f'Superfecta races:
{df[df.winning_numbers.notna()].groupby(["track_id",
"race_date", "race_number"]).size().count():,}')
"
```

Phase 2: Model Development

Shell

```
# 1. Initial training with full dataset
python train_test_residual.py \
    --csv historical_data.csv \
    --output training_results.csv

# 2. Hyperparameter optimization
python tune_residual.py \
    --csv historical_data.csv \
    --trials 100 \
    --major-tracks-only

# 3. Validate final model performance
```

```
python train_test_residual.py \  
    --csv historical_data.csv \  
    --sample 5000 \  
    --output validation_results.csv
```

Phase 3: Production Deployment

Shell

```
# 1. Test with historical race  
python predict_single_race.py \  
    --input single_race_test.csv \  
    --model  
anti_overfit_model_full_anti_overfit_major_tracks.json \  
    --output test_predictions.csv \  
    --top-combinations 10  
  
# 2. Live predictions  
python model_main.py \  
    --model  
anti_overfit_model_full_anti_overfit_major_tracks.json \  
    --output live_predictions.csv \  
    --top-combinations 20
```

Live Predictions

NYRA API Integration

The system integrates with NYRA's real-time API for live race predictions:

Shell

```
# Interactive mode (select races manually)  
python model_main.py --model your_model.json  
  
# Automated mode (first available race)
```

```
python model_main.py --model your_model.json --auto-select
```

```
# Specific race by ID
```

```
python model_main.py --model your_model.json --race-id  
"BEL123_R5"
```

Live prediction workflow:

1. Fetches today's race cards from NYRA
2. Displays available races for selection
3. Gets real-time betting pool data
4. Calculates market-implied probabilities
5. Applies ML model corrections
6. Outputs ranked superfecta combinations

Troubleshooting

Common Issues

1. "No valid races found"

Shell

```
# Check your CSV format
```

```
python -c "
```

```
import pandas as pd
```

```
df = pd.read_csv('your_data.csv', nrows=5)
```

```
print(df.columns.tolist())
```

```
print(df.head())
```

```
"
```

```
# Verify required columns exist
```

```
# Required: track_id, race_date, race_number, odds,  
winning_numbers, etc.
```

2. "Module not found" errors

Shell

```
# Reinstall packages
pip install --force-reinstall pandas numpy xgboost scikit-learn

# Check Python environment
python -c "import sys; print(sys.executable)"
```

3. NYRA API connection issues

Shell

```
# Test API connectivity
python -c "
from list_cards import list_cards
try:
    cards = list_cards()
    print(f'✅ API working, found {len(cards)} cards')
except Exception as e:
    print(f'❌ API error: {e}')
"
```

4. Model performance issues

- Ensure you have sufficient training data (50,000+ races recommended)
- Use hyperparameter optimization for better results
- Check for data quality issues (missing odds, invalid payouts)

Performance Optimization

For large datasets:

Shell

```
# Use sampling during development
python train_test_residual.py --csv data.csv --sample 100000

# Use multiple CPU cores for hyperparameter tuning
python tune_residual.py --csv data.csv --trials 50 # (adjust
based on CPU)
```

Memory management:

Shell

```
# Monitor memory usage
python -c "
import psutil
print(f'Available RAM: {psutil.virtual_memory().available /
1024**3:.1f} GB')
"
```

Expected Results

Model Performance Targets:

- **RMSE Improvement:** 15-25% over baseline Harville
- **R² Score:** 0.65-0.85 depending on data quality
- **Overfitting Gap:** <0.10 for production models